



MSC. DATA SCIENCE

Eigenfaces

Authors:

Aarcha Mukesh MDS202501
Aarushi Kumar MDS202502
Abhisri M MDS202503
Agnivesh Chatterjee MDS202504

Supervisor:

Priyavrat Deshpande
Professor, Chennai
Mathematical Institute
pdeshpande@cmi.ac.in

PROJECT REPORT

May 2026

Contributions

Team Member	Contributions
Aarcha	Prepared the Olivetti faces data, structuring the specific training and testing subsets required for evaluation.
Aarushi	Detailed the mathematical foundations of PCA and its application to Eigenfaces and formatted the final report.
Abhisri	Python implementation from first principles, utilizing NumPy for efficient vectorized matrix computations.
Agnivesh	Translated the mathematical framework into procedural algorithm steps and formatted the final report.

Table 1: Summary of Project Contributions by Team Members

Contents

1	Introduction	5
2	Dataset Description	5
3	Mathematical Background	6
3.1	Image Representation	7
3.2	Mean Face	7
3.3	Covariance Matrix and PCA	7
3.4	The $M \times M$ Trick	7
3.5	Eigenfaces and Face Space	8
4	Algorithm	8
4.1	Training Phase	8
4.1.1	Step 1: Data Preparation and Vectorization	8
4.1.2	Step 2: Compute the Mean Face	8
4.1.3	Step 3: Mean-Center the Data	8
4.1.4	Step 4: Construct the Data Matrix	8
4.1.5	Step 5: Compute Eigenfaces via the Reduced Covariance Matrix	9
4.1.6	Step 6: Select Top M' Eigenfaces	9
4.1.7	Step 7: Project Training Images into Face Space	9
4.1.8	Step 8: Compute Class Centroids	9
4.2	Recognition Phase	9
4.2.1	Step 1: Preprocessing and Projection	9
4.2.2	Step 2: Compute Distance to Class Centroids	9
4.2.3	Step 3: Compute Distance from Face Space	10
4.2.4	Step 4: Decision Rule	10
5	Implementation Details	10
5.1	Programming Environment and Libraries	10
5.2	Data Representation and Preprocessing	10
5.3	Efficient Computation of Eigenfaces	10
5.4	Dimensionality Reduction and Projection	11
5.5	Classification Strategy	11
5.6	Performance Evaluation and Experimentation	11
5.7	Implementation Considerations	11
6	Results	12
6.1	Mean Face	12
6.2	Eigenfaces	12
6.3	Eigenvalue Spectrum and Dimensionality Selection	13
6.4	Recognition Accuracy as a Function of M'	13
6.5	Threshold Analysis	14
6.6	Reconstruction Quality	15
6.7	Face vs. Non-Face Detection	15
7	Discussion	16
8	Limitations	17
9	Conclusion	18

10 Python Implementation Code	19
10.1 Imports and dataset	19
10.2 The mean face	20
10.3 Mean-centering and the data matrix A	20
10.4 Eigenfaces via the $M \times M$ trick	20
10.5 Pattern vectors and class centroids	21
10.6 Recognition: distance to class + distance to face space	21
10.7 Threshold θ_ϵ sweep - accuracy vs. unknown-rate tradeoff	22
10.8 Reconstruction demo	23
10.9 "Faceness" - face-space distance for face vs. non-face inputs	23

1 Introduction

Face recognition is a central problem in computer vision, with applications in security, authentication, and human-computer interaction. Despite its apparent simplicity for humans, automatic face recognition is challenging due to variations in illumination, facial expressions, pose, and background. These variations can significantly alter pixel-level representations of images, making direct comparison in the original image space unreliable.

Early approaches to face recognition relied on handcrafted features and heuristic methods, which often struggled to generalize across different conditions. A more principled approach is to treat images as high-dimensional data and apply statistical techniques to identify meaningful patterns. In this context, Principal Component Analysis (PCA) provides a natural framework for reducing dimensionality while preserving the most significant variations in the data.

The Eigenfaces method, introduced by Matthew Turk and Alex Pentland, applies PCA to a collection of face images to construct a low-dimensional subspace known as *face space*. The principal components of this space, called eigenfaces, represent the directions along which facial images vary the most. By projecting images onto this subspace, each face can be represented compactly as a set of coefficients, enabling efficient comparison and classification.

In this report, we implement the Eigenfaces method from first principles using only linear algebra operations. The goal is not only to reproduce the original algorithm but also to analyze its behavior under different choices of subspace dimension and classification criteria. Using the Olivetti faces dataset, we evaluate recognition accuracy, study reconstruction quality, and examine the trade-offs inherent in dimensionality reduction.

This report presents a from-scratch implementation of the Eigenfaces method for face recognition, originally proposed by Matthew Turk and Alex Pentland. The approach models facial images as high-dimensional vectors and applies Principal Component Analysis (PCA) to derive a low-dimensional subspace, referred to as *face space*. The principal components, or eigenfaces, capture the dominant modes of variation across the dataset and enable compact representation of facial features.

Using the Olivetti faces dataset, we construct eigenfaces via the computationally efficient $M \times M$ covariance trick and perform classification by comparing projected representations in face space. We evaluate recognition accuracy as a function of the number of eigenfaces retained and analyze the trade-off between dimensionality and performance. Additional experiments demonstrate reconstruction quality and the ability of the model to distinguish face and non-face inputs. The results highlight both the effectiveness and the limitations of linear subspace methods for face recognition under controlled conditions.

2 Dataset Description

The experiments in this report are conducted using the Olivetti face dataset, a widely used benchmark for evaluating face recognition algorithms. The dataset consists of 400 grayscale images of 40 distinct individuals, with 10 images per subject. Each image has a resolution of 64×64 pixels, resulting in a 4096-dimensional vector when flattened.

A key characteristic of this dataset is that it was collected under **controlled conditions**. The faces are approximately aligned, centered, and captured against a uniform background, with limited variations in pose and lighting. While there are some changes in facial expression (such as smiling or neutral faces) and minor lighting differences, the overall variability is relatively constrained compared to real-world datasets.

This controlled nature makes the dataset particularly suitable for evaluating subspace methods like Eigenfaces, as it aligns well with the algorithm’s assumptions of consistent scale, orientation, and illumination. At the same time, the dataset still contains enough intra-class variation to test the robustness of the method.

Sample face from each of 32 subjects



For this implementation, the dataset is split into training and test sets by selecting 7 images per subject for training and 3 for evaluation, yielding 280 training images and 120 test images. This split ensures that each individual is represented in both sets while maintaining a sufficient number of samples for learning the face space.

3 Mathematical Background

Before presenting the mathematical formulation of our approach, we first summarize the notation used throughout this report. Keeping track of symbols is essential, as we found notation to be one of the main challenges when studying the original literature.

Symbol	Meaning	Shape
Γ_i	i -th face image as a vector	N^2
Ψ	Mean face	N^2
$\Phi_i = \Gamma_i - \Psi$	Mean-centered face	N^2
$A = [\Phi_1 \ \Phi_2 \ \dots \ \Phi_M]$	Data matrix	$N^2 \times M$
$C = 1MAA^\top$	Covariance matrix	$N^2 \times N^2$
$L = A^\top A$	Reduced Gram matrix	$M \times M$
$\mathbf{v}_l$	Eigenvector of L	M
$\mathbf{u}_l$	Eigenface	N^2
$\omega_k = \mathbf{u}_k^\top (\Gamma - \Psi)$	Weight	Scalar
$\Omega = [\omega_1, \dots, \omega_{M'}]^\top$	Pattern vector	M'
$\varepsilon_k^2 = \Omega - \Omega_k^2$	Distance to class k	Scalar
$\varepsilon^2 = \Phi - \Phi_f^2$	Distance from face space	Scalar

Table 2: Notation used in the Eigenfaces framework

3.1 Image Representation

In the Eigenfaces framework, each grayscale image is treated as a vector in a high-dimensional Euclidean space. Given an image of size $H \times W$, it is reshaped into a vector in R^{N^2} , where $N^2 = H \times W$. For the Olivetti dataset, this corresponds to $64 \times 64 = 4096$ dimensions per image.

Formally, each face image is represented as:

$$\Gamma_i \in R^{N^2}, \quad i = 1, 2, \dots, M$$

This representation allows us to interpret the dataset as a collection of points in a high-dimensional space. However, working directly in this space is computationally inefficient and often unnecessary, since many dimensions are highly correlated (e.g., neighboring pixels). The key idea is that face images lie near a much lower-dimensional subspace.

3.2 Mean Face

To apply PCA, the dataset must first be centered. The mean face is defined as:

$$\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i$$

This mean face captures the common structure shared across all images. Each image is then centered:

$$\Phi_i = \Gamma_i - \Psi$$

This ensures that principal components capture directions of maximum variance rather than being biased by absolute position.

3.3 Covariance Matrix and PCA

Principal Component Analysis seeks directions of maximum variance, given by eigenvectors of the covariance matrix:

$$C = \frac{1}{M} A A^\top$$

where

$$A = [\Phi_1, \Phi_2, \dots, \Phi_M] \in R^{N^2 \times M}$$

The covariance matrix $C \in R^{N^2 \times N^2}$ encodes how pixel intensities vary together. Its eigenvectors define principal directions, and eigenvalues quantify variance along those directions.

However, since N^2 is large (e.g., 4096), direct computation is expensive.

3.4 The $M \times M$ Trick

Instead of diagonalizing C , we compute eigenvectors of:

$$L = A^\top A \in R^{M \times M}$$

Let v_l satisfy:

$$A^\top A v_l = \mu_l v_l$$

Multiplying by A :

$$A A^\top (A v_l) = \mu_l (A v_l)$$

Thus, $u_l = A v_l$ are eigenvectors of C . These are normalized as:

$$u_l \leftarrow \frac{u_l}{\|u_l\|}$$

Since $M \ll N^2$, this method is computationally efficient. Only $M - 1$ eigenvectors have non-zero eigenvalues due to centering.

3.5 Eigenfaces and Face Space

The eigenvectors u_i are called *eigenfaces*. They represent dominant modes of variation in the dataset.

Selecting the top M' eigenfaces defines a lower-dimensional *face space*.

Projection of a centered image Φ :

$$\omega_k = u_k^\top \Phi, \quad k = 1, 2, \dots, M'$$

Representation in face space:

$$\Omega = [\omega_1, \omega_2, \dots, \omega_{M'}]^\top$$

This provides a compact and discriminative representation.

Reconstruction from face space:

$$\Phi_f = \sum_{k=1}^{M'} \omega_k u_k = U \Omega$$

$$\Gamma \approx \Psi + \Phi_f$$

This reconstruction shows how well the subspace captures the original image.

4 Algorithm

The Eigenfaces method consists of two main stages: a **training phase**, where the face space is constructed, and a **recognition phase**, where new images are classified. Each step is described in detail below.

4.1 Training Phase

The goal of the training phase is to compute a low-dimensional subspace (face space) that captures the principal variations among facial images.

4.1.1 Step 1: Data Preparation and Vectorization

Each training image is converted from an $H \times W$ grayscale image into a vector $\Gamma_i \in R^{N^2}$, where $N^2 = H \cdot W$. The entire training set of M images is thus represented as:

$$\{\Gamma_1, \Gamma_2, \dots, \Gamma_M\}$$

4.1.2 Step 2: Compute the Mean Face

The mean face is computed as:

$$\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i$$

4.1.3 Step 3: Mean-Center the Data

Each image is centered by subtracting the mean face:

$$\Phi_i = \Gamma_i - \Psi$$

4.1.4 Step 4: Construct the Data Matrix

The centered images are arranged as columns of the data matrix:

$$A = [\Phi_1, \Phi_2, \dots, \Phi_M] \in R^{N^2 \times M}$$

4.1.5 Step 5: Compute Eigenfaces via the Reduced Covariance Matrix

Instead of directly computing the covariance matrix $C = AA^\top$, which is large ($N^2 \times N^2$), we compute:

$$L = A^\top A \in R^{M \times M}$$

We then solve the eigenvalue problem:

$$Lv_l = \mu_l v_l$$

Each eigenvector v_l is mapped back to the original space:

$$u_l = Av_l$$

Finally, each u_l is normalized:

$$u_l \leftarrow \frac{u_l}{\|u_l\|}$$

These normalized vectors u_l are the **eigenfaces**.

4.1.6 Step 6: Select Top M' Eigenfaces

The eigenfaces are sorted in descending order of eigenvalues. The top M' eigenfaces form the basis of face space:

$$U = [u_1, u_2, \dots, u_{M'}]$$

4.1.7 Step 7: Project Training Images into Face Space

Each centered training image is projected:

$$\Omega_i = U^\top (\Gamma_i - \Psi)$$

4.1.8 Step 8: Compute Class Centroids

For each class k , the centroid is computed as:

$$\Omega_k = \frac{1}{n_k} \sum_{i \in class k} \Omega_i$$

4.2 Recognition Phase

Given a new input image Γ , the goal is to determine its identity or reject it.

4.2.1 Step 1: Preprocessing and Projection

The input image is mean-centered:

$$\Phi = \Gamma - \Psi$$

Projection into face space:

$$\Omega = U^\top \Phi$$

4.2.2 Step 2: Compute Distance to Class Centroids

Distance to each class centroid:

$$\varepsilon_k = \|\Omega - \Omega_k\|$$

The class with the smallest ε_k is the closest match.

4.2.3 Step 3: Compute Distance from Face Space

Reconstruction:

$$\Phi_f = U\Omega$$

Reconstruction error:

$$\varepsilon = \|\Phi - \Phi_f\|$$

4.2.4 Step 4: Decision Rule

The classification depends on two thresholds: θ_ε and θ_ξ .

- If $\varepsilon < \theta_\xi$ and $\varepsilon_k < \theta_\varepsilon$: Recognized as class k
- If $\varepsilon < \theta_\xi$ and $\varepsilon_k \geq \theta_\varepsilon$: Unknown face
- If $\varepsilon \geq \theta_\xi$: Not a face

5 Implementation Details

The Eigenfaces algorithm is implemented entirely from first principles using Python, with a focus on clarity, mathematical fidelity, and computational efficiency. The implementation closely follows the original formulation by Turk and Pentland, avoiding the use of high-level PCA utilities in order to explicitly demonstrate each step of the method.

5.1 Programming Environment and Libraries

The core implementation is carried out using **NumPy**, which provides efficient support for vectorized operations and linear algebra routines. All major computations—including matrix multiplication, eigenvalue decomposition, and projections—are performed using NumPy functions.

Matplotlib is used for visualization of results, including eigenfaces, reconstruction images, and performance plots. Additionally, **scikit-learn** is used solely for loading the Olivetti dataset; it is not used for any part of the PCA computation or classification pipeline.

5.2 Data Representation and Preprocessing

All images are converted to double-precision floating-point format and normalized to lie in the range $[0, 1]$. Each 64×64 image is reshaped into a vector of dimension 4096, enabling the dataset to be represented as:

$$X \in R^{M \times 4096}$$

The dataset is partitioned into training and testing subsets using a per-class split (7 training and 3 testing images per subject). This ensures that each class is well-represented in both sets and prevents class imbalance.

Mean-centering is performed by subtracting the average face vector from all training and test images. This step is implemented efficiently using broadcasting, avoiding explicit loops.

5.3 Efficient Computation of Eigenfaces

A central challenge in PCA-based methods is the high dimensionality of the covariance matrix. Instead of directly computing the 4096×4096 covariance matrix, the implementation uses the reduced $M \times M$ matrix:

$$L = A^\top A$$

Eigenvalue decomposition is performed on this smaller matrix using `np.linalg.eigh`, which is optimized for symmetric matrices. The resulting eigenvectors are sorted in descending order of eigenvalues and mapped back to the original space via matrix multiplication with the data matrix.

The computed eigenfaces are then normalized to unit length to ensure orthonormality. Degenerate eigenvectors corresponding to near-zero eigenvalues are discarded to maintain numerical stability.

5.4 Dimensionality Reduction and Projection

A subset of the leading eigenfaces is selected to form the projection matrix U . The number of components M' is treated as a tunable parameter and is varied in experiments to study its effect on performance.

Projection of images into face space is implemented as:

$$\Omega = (X - \Psi)U$$

This vectorized formulation allows efficient computation of projections for entire batches of images simultaneously.

5.5 Classification Strategy

Classification is performed using a nearest-centroid approach in the reduced feature space. For each class, a centroid is computed as the mean of its projected training vectors.

Given a test image, its projection is compared to all class centroids using Euclidean distance. The predicted label corresponds to the nearest centroid.

Additionally, two distance measures are computed:

- Distance to the nearest class centroid (for identity recognition)
- Distance from face space (for detecting non-face inputs)

These are used to implement a threshold-based decision rule for rejecting unknown or non-face inputs.

5.6 Performance Evaluation and Experimentation

The implementation includes multiple experiments to evaluate the behavior of the algorithm:

- **Accuracy vs. number of eigenfaces (M'):** Study the effect of dimensionality on recognition performance
- **Threshold analysis:** Examine trade-offs between accuracy and rejection rate
- **Reconstruction experiments:** Evaluate approximation quality using different numbers of eigenfaces
- **Face vs. non-face testing:** Use random noise inputs to test robustness

All experiments are implemented using reusable functions and vectorized operations, ensuring both clarity and computational efficiency.

5.7 Implementation Considerations

Several practical considerations are addressed:

- **Numerical stability:** Small eigenvalues are filtered out
- **Efficiency:** Fully vectorized matrix operations
- **Reproducibility:** Fixed random seeds for consistent splits

- **Modularity:** Reusable functions for projection, classification, and evaluation

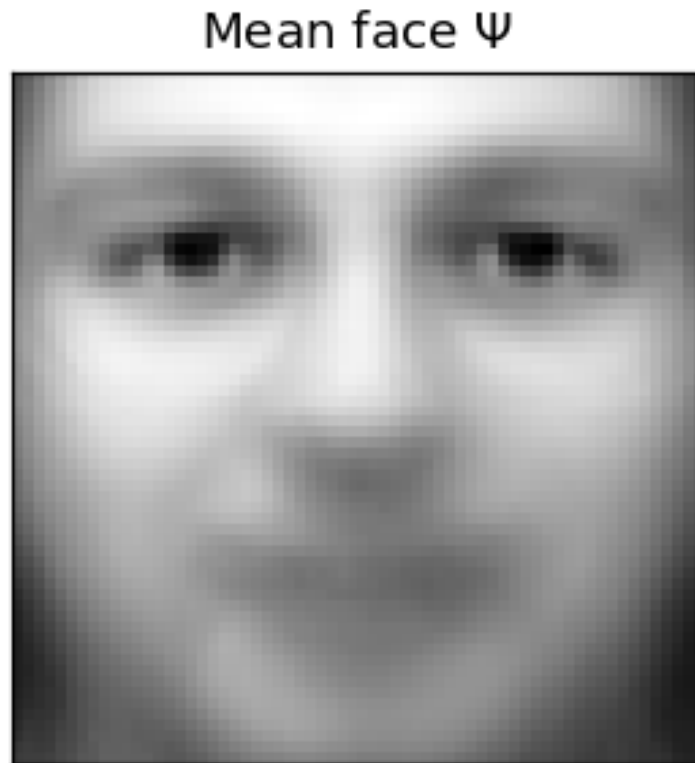
Overall, the implementation balances mathematical transparency with computational practicality, providing a faithful reproduction of the Eigenfaces algorithm.

6 Results

This section presents the experimental outcomes of the Eigenfaces implementation, highlighting both qualitative and quantitative aspects of performance. The results closely follow the structure of the original method, demonstrating how dimensionality reduction via PCA enables effective face representation and recognition.

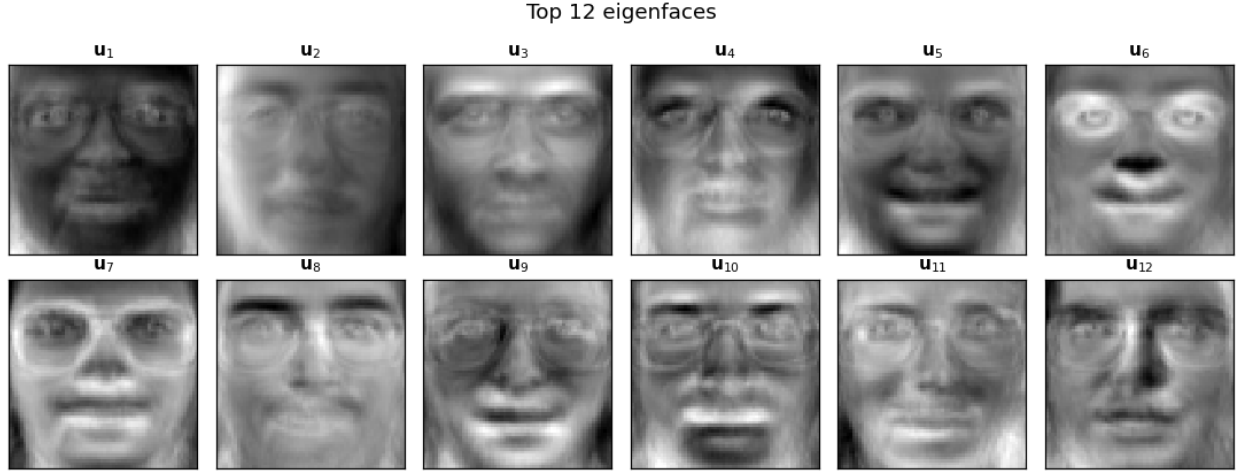
6.1 Mean Face

The mean face, computed as the average of all training images, represents the common structure shared across the dataset. As expected, the resulting image appears blurred, with prominent features such as eyes, nose, and mouth faintly visible. This reflects the averaging of individual variations and serves as the reference point for all subsequent computations.



6.2 Eigenfaces

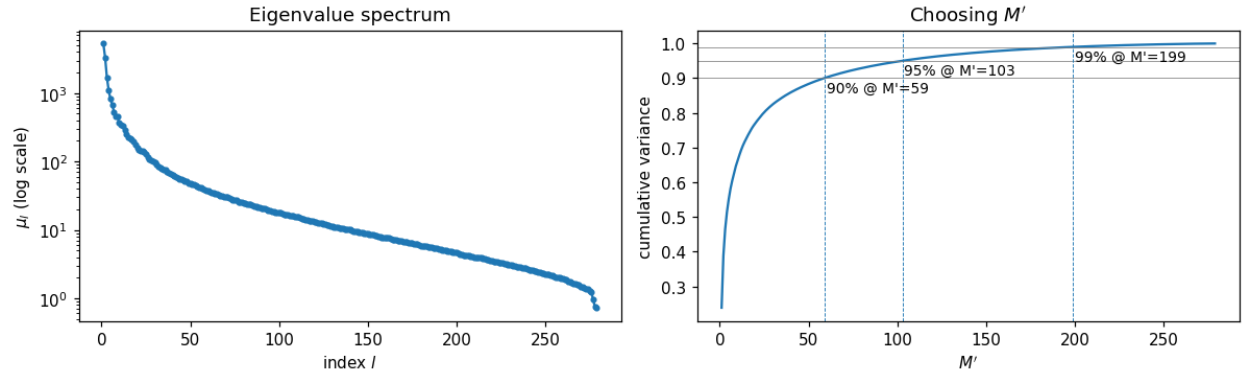
The top eigenfaces correspond to the principal directions of variation in the dataset. When visualized, these eigenfaces appear as ghost-like images, emphasizing regions where pixel intensities vary the most across different individuals. The first few eigenfaces typically capture large-scale variations such as lighting direction and overall facial structure, while higher-order eigenfaces capture finer details.



6.3 Eigenvalue Spectrum and Dimensionality Selection

The eigenvalue spectrum exhibits a rapid decay, indicating that a significant portion of the variance in the dataset is captured by a relatively small number of components. The cumulative variance curve further supports this observation, showing that a modest number of eigenfaces (e.g., 40–80) is sufficient to retain most of the information in the data.

This justifies the use of a reduced subspace for both computational efficiency and effective representation.



The paper notes that $M' < M$ eigenfaces suffice for recognition because we do not require accurate reconstruction, but only enough resolution in face space to separate identities. Sirovich and Kirby (1987) found that approximately 40 eigenfaces yielded a 2% RMS reconstruction error on 115 faces, while Turk and Pentland used $M' = 7$ on 16 training images.

We plot the eigenvalue spectrum and the cumulative explained variance to inform the choice of M' .

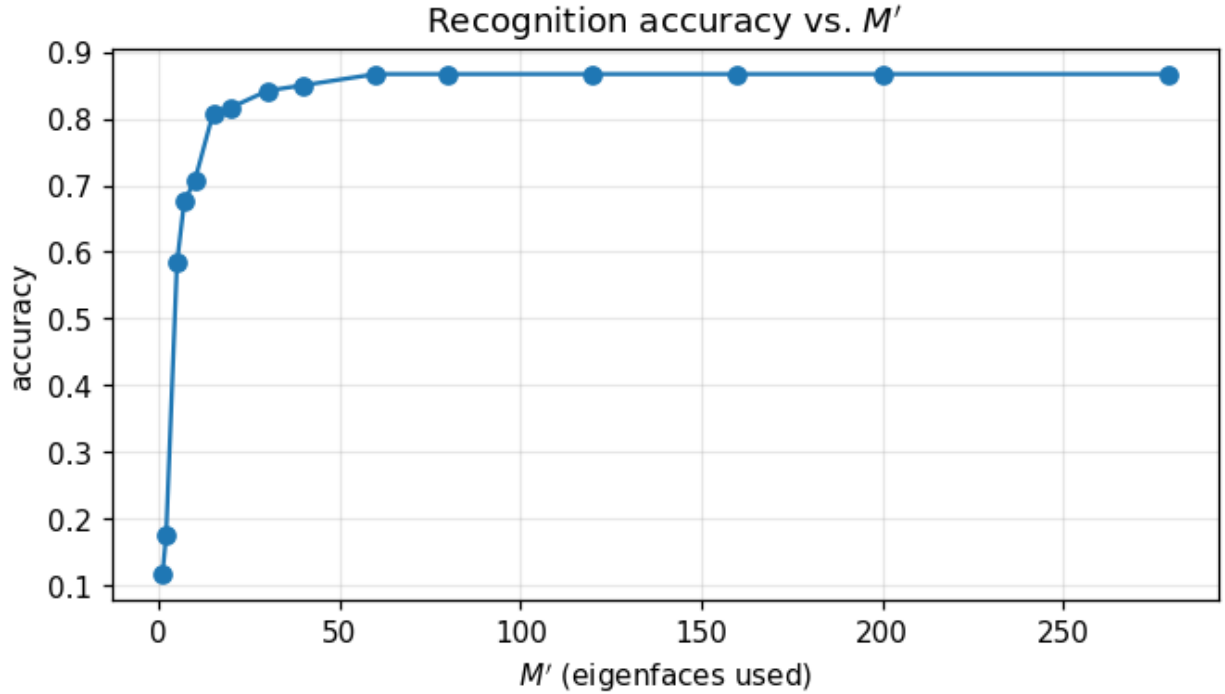
6.4 Recognition Accuracy as a Function of M'

Recognition accuracy is evaluated on the held-out test set as the number of retained eigenfaces M' is varied. The results show a clear trend:

- For very small M' , accuracy is low due to insufficient representation capacity
- As M' increases, accuracy improves significantly

- Beyond a certain point (approximately $M' = 60$), accuracy saturates around 85%–87%

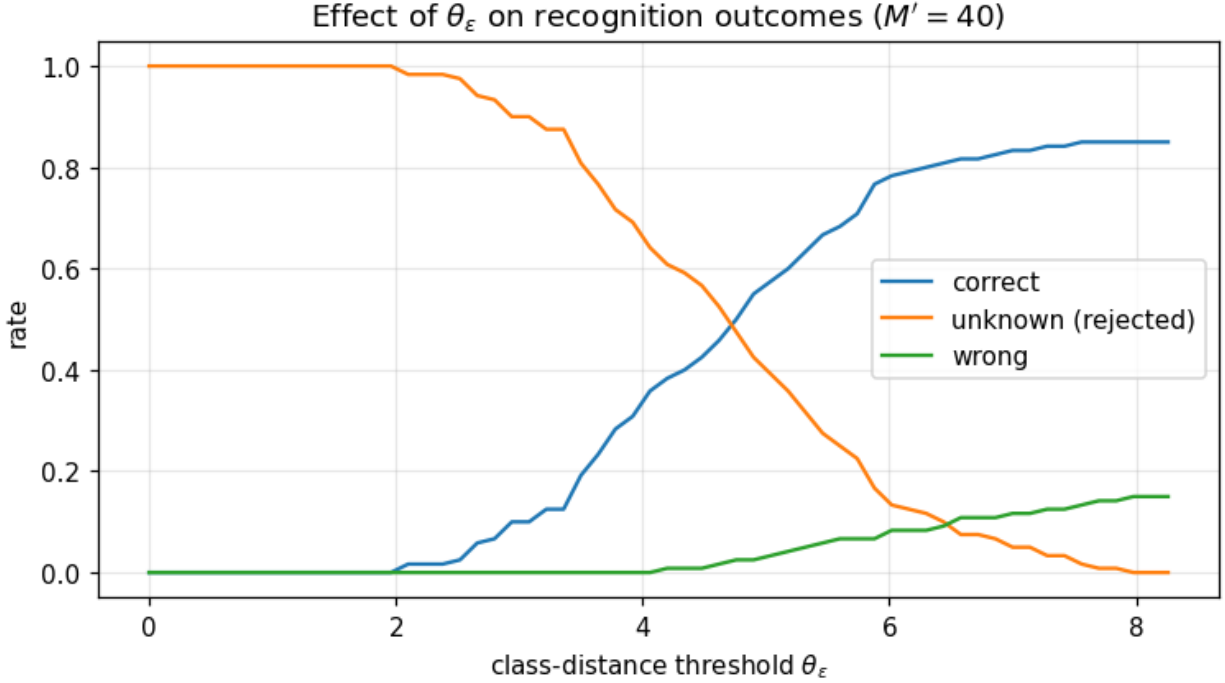
This behavior demonstrates that most discriminative information is contained in the leading principal components, and additional components provide diminishing returns.



6.5 Threshold Analysis

By varying the classification threshold, we observe a trade-off between correct classification, rejection of unknown inputs, and misclassification. Lower thresholds result in stricter acceptance criteria, reducing false positives but increasing the number of rejected inputs. Conversely, higher thresholds increase acceptance but may lead to more incorrect classifications.

This analysis highlights the importance of selecting an appropriate threshold depending on the application requirements.

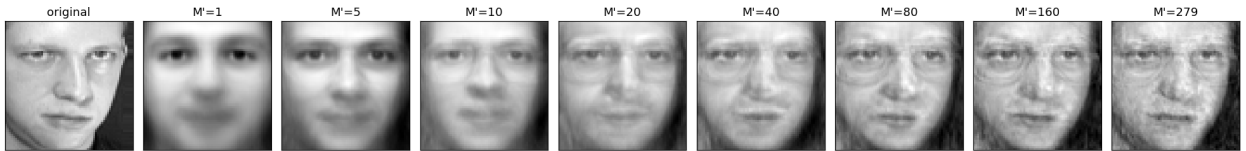


6.6 Reconstruction Quality

Reconstruction experiments illustrate how well the reduced subspace captures the original images. When a small number of eigenfaces is used, reconstructed images retain only coarse facial structure. As M' increases, finer details are progressively restored, and the reconstructed images become visually closer to the originals.

This demonstrates the ability of PCA to provide a compact yet expressive representation of facial data.

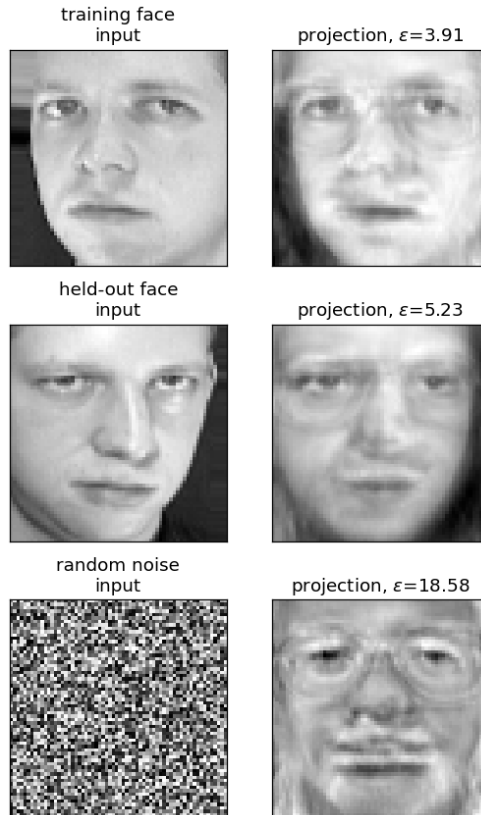
Project a held-out face into face space, then reconstruct it as $\Psi + U_{M'}\Omega$. We expect the reconstruction to appear face-like even for small M' , with progressively sharper detail as M' increases.



6.7 Face vs. Non-Face Detection

The distance from face space serves as a useful measure for distinguishing face images from non-face inputs. Genuine face images project close to the subspace and yield low reconstruction error, while non-face inputs (such as random noise) produce significantly higher errors.

This property enables the system to identify whether an input image is likely to represent a face, extending the method beyond simple classification.



Overall, the results confirm that the Eigenfaces method effectively reduces dimensionality while preserving key information necessary for recognition. The experiments also provide insight into the trade-offs between representation size, accuracy, and robustness.

7 Discussion

The results demonstrate that the Eigenfaces method provides an effective and computationally efficient approach to face recognition under controlled conditions. By projecting high-dimensional image data onto a lower-dimensional subspace, the algorithm captures the most significant variations in facial structure while discarding redundant information. This allows for both compact representation and relatively accurate classification using simple distance-based methods.

One of the key observations from the results is the rapid improvement in recognition accuracy as the number of eigenfaces M' increases, followed by a clear saturation point. This indicates that most of the discriminative information required for classification is contained within a relatively small number of principal components. Beyond this point, adding more components yields diminishing returns, as they primarily capture noise or minor variations rather than meaningful identity-related features.

The eigenvalue spectrum further supports this observation, showing that variance in the dataset is heavily concentrated in the leading components. This justifies the use of dimensionality reduction not only for computational efficiency but also for improved generalization, as it reduces the risk of overfitting to irrelevant details.

The reconstruction experiments provide additional insight into the nature of the learned subspace. Even with a small number of eigenfaces, reconstructed images retain recognizable facial structure, suggesting that

the face space captures the essential geometry of faces. As more components are included, finer details are recovered, illustrating the progressive refinement of the representation.

The threshold analysis highlights an important practical consideration: the trade-off between recognition accuracy and robustness. A stricter threshold reduces misclassification but increases the likelihood of rejecting valid inputs, while a more lenient threshold improves acceptance at the cost of higher error rates. This trade-off must be carefully balanced depending on the intended application.

However, the effectiveness of the method is closely tied to the nature of the dataset. Since the Olivetti dataset consists of well-aligned images captured under controlled conditions, the algorithm performs well. In more realistic scenarios with significant variations in lighting, pose, and background, performance would likely degrade. This limitation arises from the linear nature of PCA, which cannot fully capture complex, nonlinear variations in facial appearance.

Overall, the Eigenfaces method illustrates the power of linear subspace techniques for high-dimensional data analysis. While it may not be sufficient for modern, unconstrained face recognition tasks, it remains an important foundational approach that provides valuable insight into the role of dimensionality reduction in pattern recognition.

8 Limitations

While the Eigenfaces method provides an elegant and efficient framework for face recognition, it has several important limitations that restrict its applicability in real-world scenarios.

A primary limitation is its **sensitivity to variations in illumination**. Since PCA captures directions of maximum variance in pixel intensity, changes in lighting conditions can dominate the principal components, often overshadowing identity-specific features. As a result, the same individual under different lighting conditions may be represented quite differently in face space, leading to misclassification.

Another significant limitation is the assumption of **well-aligned and centered faces**. The method relies on pixel-wise correspondence across images, meaning that facial features such as the eyes, nose, and mouth must appear in approximately the same locations in every image. Even small misalignments, rotations, or scale differences can introduce large variations in the vectorized representation, negatively affecting performance.

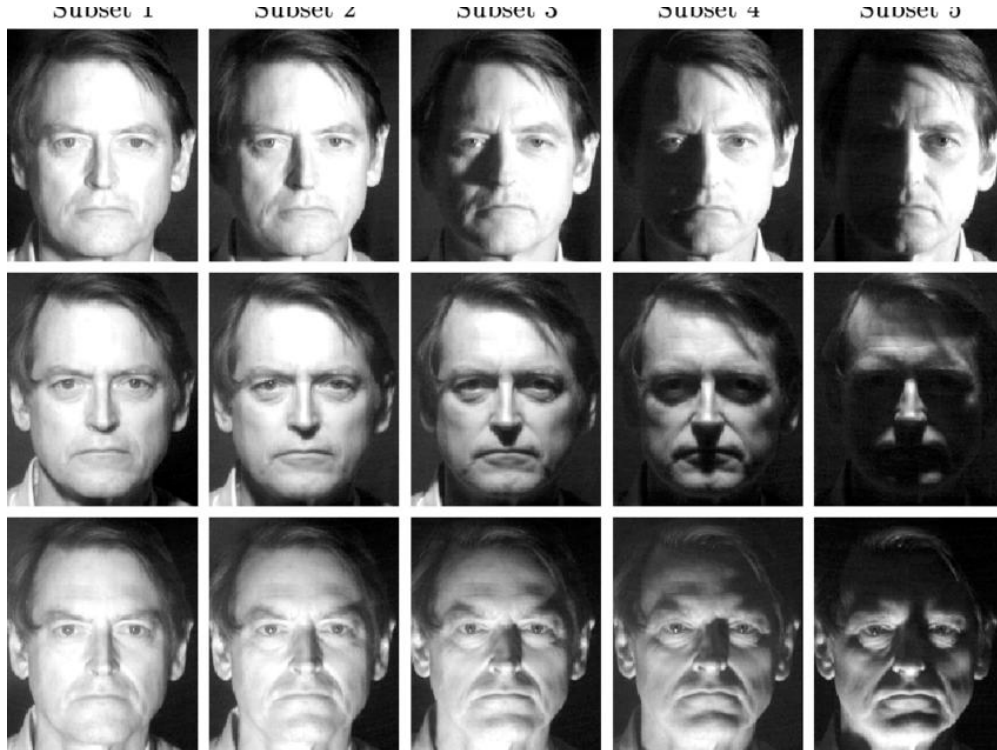
The method is also inherently **linear**, as it models faces using a linear combination of eigenfaces. This restricts its ability to capture complex, nonlinear variations in facial appearance, such as changes in expression, pose, or occlusion. Real-world facial data often lies on a nonlinear manifold, which cannot be fully represented within a linear subspace.

In addition, the approach is **sensitive to noise and outliers**. Since PCA minimizes reconstruction error in a least-squares sense, it can be influenced by anomalous images in the training set, potentially distorting the learned subspace.

Another practical limitation is that the method does not inherently handle **unknown or unseen identities** in a robust manner. While thresholding can be used to reject inputs that do not match known classes, selecting appropriate thresholds is nontrivial and often dataset-dependent.

Finally, the method assumes a **fixed image resolution and consistent preprocessing pipeline**. Any deviation in image size, background, or acquisition conditions requires additional preprocessing steps, such as normalization or alignment, which are not part of the core algorithm.

These limitations highlight that, although Eigenfaces is effective in controlled environments, it lacks robustness in unconstrained settings. This has motivated the development of more advanced techniques, including nonlinear methods and deep learning-based approaches, which can better handle the complexities of real-world face recognition.



The above figure illustrates the sensitivity of the Eigenfaces method to illumination. Although all images correspond to the same individual, variations in lighting significantly alter pixel intensities, resulting in different projections in face space. In contrast, images of different individuals under similar lighting conditions may appear closer in the reduced space. This occurs because PCA captures directions of maximum variance, which are often dominated by illumination rather than identity.

9 Conclusion

In this report, we presented a complete implementation of the Eigenfaces method for face recognition, following the original formulation by Matthew Turk and Alex Pentland. By representing facial images as high-dimensional vectors and applying Principal Component Analysis, we constructed a lower-dimensional subspace—referred to as face space—that captures the most significant variations in the dataset.

The results demonstrate that effective recognition can be achieved using a relatively small number of eigenfaces, highlighting the power of dimensionality reduction in simplifying high-dimensional data. The experiments further show how recognition accuracy improves with increasing subspace dimension before reaching a saturation point, indicating that only a subset of principal components is necessary for reliable classification. Additional analyses, including reconstruction and threshold-based evaluation, provide deeper insight into the behavior and interpretability of the method.

Despite its simplicity, the Eigenfaces approach offers a clear and mathematically grounded framework for understanding face recognition. However, its performance is closely tied to controlled conditions, and its limitations in handling variations such as lighting, pose, and alignment make it less suitable for real-world applications.

Overall, this implementation reinforces the importance of linear algebra techniques in pattern recognition and serves as a foundational step toward more advanced methods in computer vision.

10 Python Implementation Code

The implementation is broken down into the following distinct sections, showcasing the data preparation, PCA operations, training, and testing utilizing Python and NumPy.

10.1 Imports and dataset

The implementation starts by setting up the dataset and the plotting defaults. The Olivetti faces dataset is loaded and split into training and test sets.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(0)
4
5 # Plot defaults
6 plt.rcParams["image.cmap"] = "gray"
7 plt.rcParams["image.interpolation"] = "nearest"
8 plt.rcParams["figure.dpi"] = 110
9
10 def load_olivetti():
11     """Load Olivetti faces. Tries sklearn first; falls back to a known mirror."""
12     try:
13         from sklearn.datasets import fetch_olivetti_faces
14         d = fetch_olivetti_faces()
15         return d.images.astype(np.float64), d.target.astype(int)
16     except Exception as e:
17         print(f"sklearn fetch failed ({e}); trying mirror...")
18         import urllib.request, scipy.io as sio, io
19         url = "https://cs.nyu.edu/~roweis/data/olivettifaces.mat"
20         with urllib.request.urlopen(url, timeout=60) as r:
21             buf = io.BytesIO(r.read())
22             m = sio.loadmat(buf)
23             faces = m["faces"].T.reshape(-1, 64, 64).astype(np.float64) / 255.0
24             targets = np.repeat(np.arange(40), 10)
25             return faces, targets
26
27 images, targets = load_olivetti()
28
29 fig, axes = plt.subplots(4, 8, figsize=(9, 5))
30 for i, ax in enumerate(axes.flat):
31     subj = i
32     idx = np.where(targets == subj)[0][0]
33     ax.imshow(images[idx])
34     ax.set_xticks([]); ax.set_yticks([])
35 fig.suptitle("Sample face from each of 32 subjects")
36 plt.tight_layout(); plt.show()
37
38 N_PER_SUBJ_TRAIN = 7
39 H, W = images.shape[1], images.shape[2]
40 N2 = H * W
41 train_idx, test_idx = [], []
42 rng = np.random.default_rng(42)
43
44 for s in np.unique(targets):
45     idx = np.where(targets == s)[0]
46     rng.shuffle(idx)
```

```

47     train_idx.extend(idx[:N_PER_SUBJ_TRAIN])
48     test_idx.extend(idx[N_PER_SUBJ_TRAIN:])
49
50 train_idx = np.array(train_idx)
51 test_idx = np.array(test_idx)
52
53 X_train = images[train_idx].reshape(-1, N2)
54 y_train = targets[train_idx]
55 X_test = images[test_idx].reshape(-1, N2)
56 y_test = targets[test_idx]

```

10.2 The mean face

The average face of the training dataset is computed.

```

1 Psi = X_train.mean(axis=0)
2
3 plt.figure(figsize=(3, 3))
4 plt.imshow(Psi.reshape(H, W))
5 plt.title(r"Mean face $\Psi$"); plt.xticks([]); plt.yticks([])
6 plt.show()

```

10.3 Mean-centering and the data matrix A

Each face deviates from the average by subtracting the mean face to build the data matrix.

```

1 Phi_train = X_train - Psi
2 A = Phi_train.T
3 print("A shape:", A.shape)

```

10.4 Eigenfaces via the $M \times M$ trick

The eigenvectors are calculated using the reduced Gram matrix and then mapped back to compute the eigenfaces. The top eigenfaces are visualized and the eigenvalue spectrum is plotted.

```

1 L = A.T @ A
2 mu_L, V = np.linalg.eigh(L)
3 order = np.argsort(mu_L)[::-1]
4 mu_L, V = mu_L[order], V[:, order]
5
6 U = A @ V
7 norms = np.linalg.norm(U, axis=0)
8 keep = norms > 1e-8
9 U = U[:, keep] / norms[keep]
10 mu = mu_L[keep]
11
12 # Visualize Top Eigenfaces
13 n_show = 12
14 fig, axes = plt.subplots(2, 6, figsize=(10, 4))
15 for i, ax in enumerate(axes.flat):
16     ax.imshow(U[:, i].reshape(H, W))
17     ax.set_title(fr"$\mathbf{u}_{\{{{i+1}\}}}$", fontsize=10)
18     ax.set_xticks([]); ax.set_yticks([])
19 fig.suptitle("Top 12 eigenfaces"); plt.tight_layout(); plt.show()

```

```

20
21 # Eigenvalue Spectrum
22 explained = mu / mu.sum()
23 cum = np.cumsum(explained)
24 fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))
25 axes[0].semilogy(np.arange(1, len(mu)+1), mu, "-")
26 axes[0].set_xlabel(r"index $l$"); axes[0].set_ylabel(r"$\mu_l$ (log scale)")
27 axes[0].set_title("Eigenvalue spectrum")
28
29 axes[1].plot(np.arange(1, len(cum)+1), cum)
30 for thr in (0.90, 0.95, 0.99):
31     k = np.searchsorted(cum, thr) + 1
32     axes[1].axhline(thr, color="gray", lw=0.5)
33     axes[1].axvline(k, ls="--", lw=0.5)
34     axes[1].text(k+1, thr-0.04, f"{int(thr*100)}% @ M'={k}", fontsize=9)
35 axes[1].set_xlabel(r"$M'$"); axes[1].set_ylabel("cumulative variance")
36 axes[1].set_title("Choosing $M'$")
37 plt.tight_layout(); plt.show()

```

10.5 Pattern vectors and class centroids

The training data is projected into the face space, and class centroids are computed for each subject.

```

1 def project(X, U_k, Psi):
2     """Project a batch of images (rows) onto the first U_k.shape[1] eigenfaces."""
3     return (X - Psi) @ U_k
4
5 def class_centroids(Omega, y):
6     classes = np.unique(y)
7     return classes, np.stack([Omega[y == c].mean(axis=0) for c in classes])
8
9 Mp = 40
10 U_k = U[:, :Mp]
11 Omega_train = project(X_train, U_k, Psi)
12 classes, Omega_centroids = class_centroids(Omega_train, y_train)

```

10.6 Recognition: distance to class + distance to face space

New images are classified by calculating their distance to known class centroids and their reconstruction distance from the face space. A sweep is performed to evaluate accuracy across different values of M' .

```

1 def classify(X, U_k, Psi, Omega_centroids, classes, theta_class=np.inf, theta_face
= np.inf):
2     Phi = X - Psi
3     Om = Phi @ U_k
4     eps = np.sqrt(np.maximum(
5         np.einsum("ij, ij->i", Phi, Phi) - np.einsum("ij, ij->i", Om, Om), 0.0))
6
7     diff = Om[:, None, :] - Omega_centroids[None, :, :]
8     d = np.linalg.norm(diff, axis=2)
9     eps_k = d.min(axis=1)
10
11     nearest = classes[d.argmin(axis=1)]
12     pred = nearest.copy()

```

```

13     pred[(eps > theta_face)] = -2
14     pred[(eps <= theta_face) & (eps_k >= theta_class)] = -1
15     return pred, eps_k, eps
16
17 pred, eps_k, eps = classify(X_test, U_k, Psi, Omega_centroids, classes)
18
19 # Accuracy as a function of M'
20 Mps = [1, 2, 5, 7, 10, 15, 20, 30, 40, 60, 80, 120, 160, 200, U.shape[1]]
21 accs = []
22 for k in Mps:
23     Uk = U[:, :k]
24     Om_tr = project(X_train, Uk, Psi)
25     cls, cent = class_centroids(Om_tr, y_train)
26     pr, _, _ = classify(X_test, Uk, Psi, cent, cls)
27     accs.append((k, (pr == y_test).mean()))
28
29 ks = [a[0] for a in accs]; vs = [a[1] for a in accs]
30 plt.figure(figsize=(7, 3.5))
31 plt.plot(ks, vs, "o-")
32 plt.xlabel(r"$M'$ (eigenfaces used)"); plt.ylabel("accuracy")
33 plt.title(r"Recognition accuracy vs. $M'$")
34 plt.grid(alpha=0.3); plt.show()

```

10.7 Threshold θ_ϵ sweep - accuracy vs. unknown-rate tradeoff

Varying the class-distance threshold affects the rates of correct classifications, rejected inputs, and wrong classifications.

```

1 Uk = U[:, :40]
2 Om_tr = project(X_train, Uk, Psi)
3 cls, cent = class_centroids(Om_tr, y_train)
4
5 _, eps_k_test, _ = classify(X_test, Uk, Psi, cent, cls)
6 thetas = np.linspace(0, eps_k_test.max() * 1.05, 60)
7
8 correct, unknown, wrong = [], [], []
9
10 for th in thetas:
11     pr, _, _ = classify(X_test, Uk, Psi, cent, cls, theta_class=th, theta_face=np.
12         inf)
13     is_unknown = pr == -1
14     is_correct = (pr == y_test) & ~is_unknown
15     is_wrong = (pr != y_test) & ~is_unknown
16     n = len(y_test)
17     correct.append(is_correct.sum()/n)
18     unknown.append(is_unknown.sum()/n)
19     wrong.append(is_wrong.sum()/n)
20
21 plt.figure(figsize=(8, 4))
22 plt.plot(thetas, correct, label="correct")
23 plt.plot(thetas, unknown, label="unknown (rejected)")
24 plt.plot(thetas, wrong, label="wrong")
25 plt.xlabel(r"class-distance threshold $\theta_\epsilon$")
26 plt.ylabel("rate"); plt.legend(); plt.grid(alpha=0.3)
27 plt.title(r"Effect of $\theta_\epsilon$ on recognition outcomes ($M'=40$)")

```

```
27 plt.show()
```

10.8 Reconstruction demo

A held-out face is projected into face space and progressively reconstructed as M' increases.

```
1 gamma = X_test[0]
2 phi = gamma - Psi
3 Mp_grid = [1, 5, 10, 20, 40, 80, 160, U.shape[1]]
4
5 fig, axes = plt.subplots(1, len(Mp_grid) + 1, figsize=(2*(len(Mp_grid)+1), 2.5))
6 axes[0].imshow(gamma.reshape(H, W)); axes[0].set_title("original")
7 axes[0].set_xticks([]); axes[0].set_yticks([])
8
9 for ax, k in zip(axes[1:], Mp_grid):
10     om = U[:, :k].T @ phi
11     rec = Psi + U[:, :k] @ om
12     ax.imshow(rec.reshape(H, W))
13     ax.set_title(f"M'={k}")
14     ax.set_xticks([]); ax.set_yticks([])
15
16 plt.tight_layout(); plt.show()
```

10.9 "Faceness" - face-space distance for face vs. non-face inputs

The method distinguishes between genuine face inputs and non-face inputs by measuring reconstruction error.

```
1 def project_and_distance(gamma, Uk, Psi):
2     phi = gamma - Psi
3     om = Uk.T @ phi
4     phi_f = Uk @ om
5     eps = np.linalg.norm(phi - phi_f)
6     return phi_f + Psi, eps
7
8 Uk = U[:, :40]
9 inputs = [
10     ("training face", X_train[0]),
11     ("held-out face", X_test[0]),
12     ("random noise", np.random.default_rng(1).random(N2)),
13 ]
14
15 fig, axes = plt.subplots(len(inputs), 2, figsize=(5, 2.6*len(inputs)))
16 for row, (label, g) in enumerate(inputs):
17     rec, eps = project_and_distance(g, Uk, Psi)
18     axes[row, 0].imshow(g.reshape(H, W))
19     axes[row, 1].imshow(rec.reshape(H, W))
20     for ax in axes[row]:
21         ax.set_xticks([]); ax.set_yticks([])
22     axes[row, 0].set_title(f"{label}\ninput")
23     axes[row, 1].set_title(rf"projection, $\epsilon$={eps:.2f}")
24
25 plt.tight_layout(); plt.show()
```

References

- [1] M. Turk and A. Pentland, “Eigenfaces for Recognition,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 1991, pp. 586–591.